FUCAPE
BUSINESS SCHOOL

Compressão de Texto e de Índices Invertidos

PLN e Sistemas de Recuperação de Informação

Prof. Dr. Wagner Perin
FUCAPE Business School

Compressão de Texto e de Índices Invertidos

Leis Empíricas, Variable Byte Encoding e Códigos Bit-Level

Disciplina: PLN e Sistemas de Recuperação da Informação
Programa: Ciência de Dados para Negócios | FUCAPE
Carga Horária: 4 h/a (200 min) | **Prof.:** Prof. Dr. Wagner Perin
Unidade Curricular: UNIDADE 2 — Indexação e Estruturas



INSIGHT EXECUTIVO

Objetivo Central da Aula:
Dominar as leis de Heaps e Zipf e implementar algoritmos de compressão (d-gaps, Variable Byte e Elias) capazes de reduzir em 75% o consumo de RAM e eliminar o gargalo do barramento de dados.

Resgate da Aula 04 & O Novo Gargalo

A Barreira dos Inteiros Fixos de 32 Bits em Grandes Acervos

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Dilema Arquitetural: Na Aula 04, superamos a esparsidade das matrizes criando o Índice Invertido. Mas o que ocorre ao armazenar milhões de DocIDs com inteiros de 32 bits?

O Avanço da Aula 04 vs. A Barreira de Escala:

- **Conquista da Aula 04:** Eliminamos a varredura $O(|D|)$ com cruzamento linear $O(L_1 + L_2)$.
- **O Ponto Frágil:** Cada postagem utilizou inteiros nativos de 32 ou 64 bits (4 ou 8 bytes).
- **Grandes Acervos:** Em coleções com 100M documentos, termos populares (*"brasil"*, *"frete"*) acumulam mais de **30 milhões de postagens cada**.
- **Impacto:** Uma única palavra consome mais de **120 MB de RAM**, saturando a memória rápida!

A Dor de Negócio: O “Memory Wall”

O Custo Financeiro da Latência e Fatura de Nuvem

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Marketplace com 100M de produtos indexados enfrenta colapso de latência e explosão na fatura de nuvem AWS.

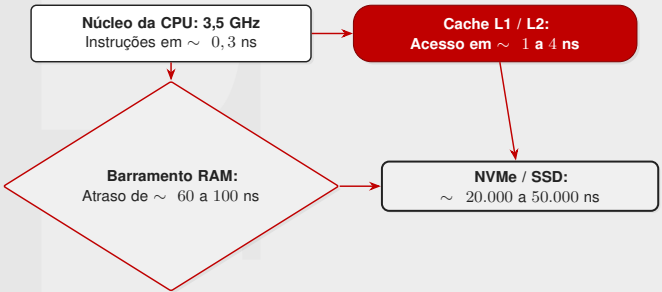
Crise de Infraestrutura e Custo:

- **Cluster AWS:** Instâncias `r5b.16xlarge` (512 GB RAM).
- **Fatura: US$ 14.800/mês** para o índice bruto em RAM.
- **Gargalo:** Latência de 120ms sob 4.000 buscas/s com núcleos ociosos esperando tráfego de memória.
- **Diagnóstico:** Esgotamento do barramento (*Memory Wall*).



O Paradoxo do Hardware Moderno

CPU GHz vs. Barramento de Memória RAM vs. NVMe



INSIGHT EXECUTIVO

A Regra de Ouro do Hardware Moderno:
A CPU é ordens de grandeza mais rápida do que a memória RAM. Transferir **menos bytes comprimidos** pelo barramento e descomprimi-los na cache é **substancialmente mais rápido** do que transferir dados brutos não-comprimidos!

O Desperdício dos Inteiros Fixos de 32 Bits

Por que Inteiros Nativos de 4 Bytes Inviabilizam Grandes Acervos?

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Se a grande maioria das distâncias entre documentos é pequena, por que gastar 32 bits (4 bytes) para inteiros que cabem em 3 ou 4 bits?

Anatomia de um Inteiro de 32 bits para o número 3:

Código Fonte

Representação em Memória (Big-Endian) [00000000] [00000000] [00000000] [00000011]
<————————- 30 bits de ZEROS inúteis ————————> <- 2 bits úteis ->

O Dilema de Escala:

- **Taxa de Desperdício:** $30/32 = \mathbf{93,75\%}$ de bits redundantes gravados na memória.
- **Em 10M postagens:** Inteiros 32-bit consomem $\mathbf{40}$ MB; em bytes variáveis consomem $\mathbf{10}$ MB ($\mathbf{75\%}$ de economia direta!).

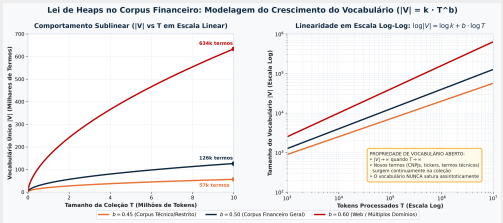
Linguística Quantitativa I: A Lei de Heaps

Vocabulário Aberto e Crescimento Sublinear do Léxico

Dilema: Como o vocabulário cresce com o corpus?

$$|\mathbf{V}| = \mathbf{k} \cdot \mathbf{T}^{\mathbf{b}}$$

- T: Total de tokens (palavras) no acervo.
- k: Parâmetro empírico ($30 \leq k \leq 100$).
- b: Expoente ($0,45 \leq b \leq 0,60$, tipicamente $\approx 0,5$).



INSIGHT EXECUTIVO

Insight de Engenharia:
O vocabulário **nunca para de crescer**. Novos documentos trazem novos termos, forçando compressão contínua do léxico!

Linguística Quantitativa II: A Lei de Zipf

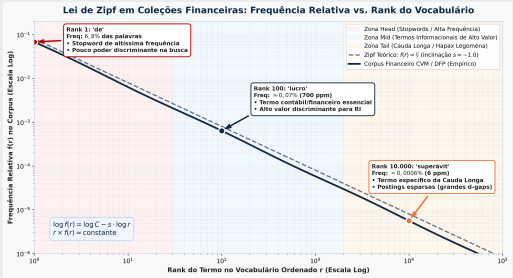
Distribuição de Potências e a Disparidade de Frequências

Formulação de Zipf: $cf_i \propto \frac{1}{i} \implies cf_i = \frac{c}{i}$

Termo	Rank (i)	Distância Média
"de"	1	≈ 10 palavras
"empresa"	10	≈ 100 palavras
"debêntures"	1.000	≈ 10.000 palavras
"superávit"	100.000	$\approx 1.000.000$ palavras

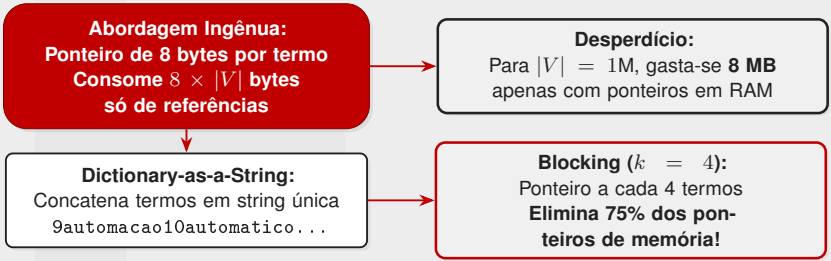
INSIGHT EXECUTIVO

Conclusão para os Índices:
Palavras frequentes geram postagens com **gaps minúsculos**; palavras raras geram **gaps grandes**.



Compressão de Dicionário em Memória (RAM)

Dictionary-as-a-String e Agrupamento em Blocos (Blocking)



Técnicas Fundamentais de Otimização do Léxico:

- **Dictionary-as-a-String:** Remove ponteiros fixos agrupando termos com prefixo de tamanho de 1 byte.
- **Blocking ($k = 4$):** Mantém ponteiros apenas para o início de cada bloco, viabilizando busca binária rápida.

Compressão de Dicionário: Front Coding

Fatoração de Prefixos Comuns em Vocabulários Ordenados

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Com vocabulário em ordem alfabética estrita, palavras consecutivas compartilham prefixos idênticos.

Exemplo Prático com 3 termos consecutivos:

Código Fonte

Entrada Original vs. Front Coding Entrada Original: [automacao] [automatico] [automovel] –> 28 caracteres Front Coding: 8auto*macao3tico3movel –> 19 caracteres

Impacto de Engenharia:

- **Fatoração de Prefixo:** O prefixo comum `"auto"` é gravado uma única vez.
- **Redução de RAM:** Economia superior a **60% no consumo textual do dicionário**.
- **Busca Preservada:** A busca binária entre blocos permanece inalterada e eficiente.

Compressão de Postings: O Conceito de d-gaps

Transformando DocIDs Absolutos em Deltas Numéricos Minúsculos

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Como transformar identificadores gigantescos (ex: DocID 850.420) em inteiros minúsculos que cabem em 1 byte?

A Regra de Ouro: Diferenças Relativas (d-gaps)

Como as postagens estão **ordenadas de forma crescente**, armazena-se a diferença:

$$d\text{-gap}_i = \text{DocID}_1 \text{ (se } i = 1) \quad | \quad d\text{-gap}_i = \text{DocID}_i - \text{DocID}_{i-1} \text{ (se } i > 1)$$

DocIDs: $[\mathbf{100.000}, \mathbf{100.004}, \mathbf{100.005}, \mathbf{100.012}, \mathbf{100.020}] \implies \mathbf{d}\text{-gaps: } [\mathbf{100.000}, \mathbf{4}, \mathbf{1}, \mathbf{7}, \mathbf{8}]$

INSIGHT EXECUTIVO

Impacto Direto: Quatro dos cinco identificadores tornaram-se números menores que 10, exigindo frações de 1 byte!

O Algoritmo Variable Byte Encoding (VB Code)

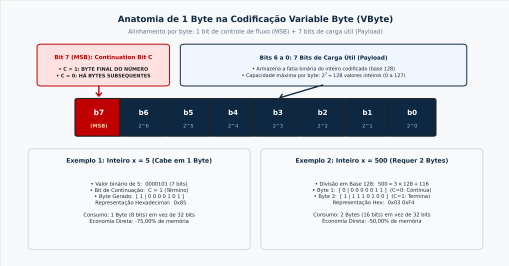
Codificação de Inteiros Alinhada a Bytes Adotada pelo Lucene

Anatomia de Cada Byte no VB Code:

- **Bit 7 (MSB) — Continuation Bit (C):**
  - $C = 1$: **Fim** do número (último byte).
  - $C = 0$: O número **continua** no próximo byte.
- **Bits 6 a 0 — 7 bits de Payload:** Armazenam o valor binário do inteiro.

Capacidade de Codificação:

- $1 \ldots 127$: **1 byte**.
- $128 \ldots 16.383$: **2 bytes**.
- $16.384 \ldots 2.097.151$: **3 bytes**.



Trace Visual da Codificação Variable Byte

Passo a Passo de Conversão Decimal $\rightarrow$ 7 Bits $\rightarrow$ Hexadecimal

Exemplo 1: Codificando $\Delta = 5$ (Número Pequeno)

- Cabe em 7 bits: $5 = 0000101_2$. Como é o único byte, liga-se o bit $C = 1$:

$$\text{Byte Gerado: } [\mathbf{1}\ 0000101_2] = \mathbf{10000101_2} = \text{0x85} \implies \mathbf{1}\ \text{byte!}$$

Exemplo 2: Codificando $\Delta = 135$ (Número Intermediário)

- Como $135 \geq 128$, dividimos por 128: $135 = 1 \times 128 + 7$.
- **Byte 1:** $135//128 = 1$. Não é o último $\implies$ $C = 0$: $[\mathbf{0}\ 0000001_2] = \text{0x01}$
- **Byte 2:** $135\%128 = 7$. É o último $\implies$ $C = 1$: $[\mathbf{1}\ 0000111_2] = \mathbf{10000111_2} = \text{0x87}$
- **Fluxo Gravado:** 0x01 0x87 $\implies$ **2** bytes!

Reflexão Analítica & Decisão Estratégica

Cálculo Prático de Taxa de Compressão e Economia de Memória

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Desafio Corporativo: Postagens de produtos com DocIDs: DocIDs $= [450, 458, 470, 610, 625]$

Resolução Analítica Guiada:

1. d**-gaps:** $[450,\ (458-450),\ (470-458),\ (610-470),\ (625-610)] = [\mathbf{450}, \mathbf{8}, \mathbf{12}, \mathbf{140}, \mathbf{15}]$
2. **Bytes em Variable Byte:** 450 (2B) + 8 (1B) + 12 (1B) + 140 (2B) + 15 (1B) = **7** bytes.
3. **Comparação de Infraestrutura:** Não-comprimido (32 bits): $5 \times 4 = \mathbf{20}$ bytes vs. Comprimido: **7** bytes $\implies \mathbf{65}\%$ de redução!

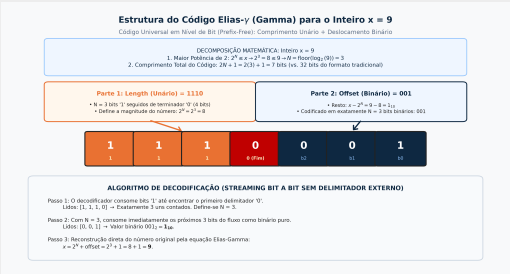
Códigos Bit-Level: Elias-Gamma (γ)

Representação Não-Alinhada com Comprimento Unário e Offset Binário

A Estrutura do Código Elias-γ: Codifica um inteiro $x \geq 1$ em nível de bit:

1. **Length (Unário):** $\lfloor \log_2 x \rfloor$ uns seguidos de zero.
2. **Offset (Binário):** $x - 2^{\lfloor \log_2 x \rfloor}$ em $\lfloor \log_2 x \rfloor$ bits.

Número (x)	Length	Offset	Elias-γ	Bits
1	0	-	**0**	1 bit
2	10	0	**100**	3 bits
3	10	1	**101**	3 bits
4	110	00	**11000**	5 bits
9	1110	001	**1110001**	7 bits



Códigos Bit-Level: Elias-Delta (δ) e Trade-offs

Densidade de Bits vs. Custo Computacional de Deslocamento de Bits

Limitação do Elias-Gamma:

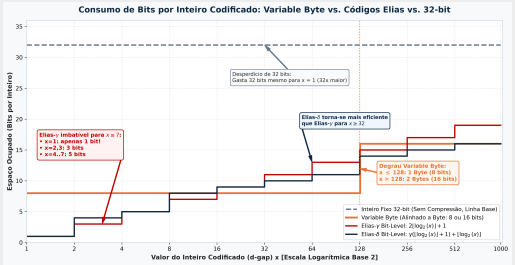
- Consome $2\lfloor \log_2 x \rfloor + 1$ bits.
- Para inteiros grandes, a parte unária cresce rápido.

A Solução: Elias-Delta (δ):

- Codifica o comprimento via Elias-γ, tornando o código ótimo para números maiores.

INSIGHT EXECUTIVO

A Decisão da Indústria (Lucene / ES):
O **Variable Byte** é preferido em busca interativa por ser **alinhado a bytes**, evitando operações bitwise lentas na CPU!



Quadro Comparativo de Engenharia de Compressão

Inteiros Fixos de 32 Bits vs. Variable Byte vs. Elias-Gamma

Métrica de Arquitetura	Inteiros Fixos (32 bits)	Variable Byte (VB Code)	Elias-Gamma (γ)
Alinhamento de Dados	Palavra de 4 bytes	**Alinhado a Byte (8 bits)**	Desalinhado (Bit-level)
Consumo Típico por Entrada	4 bytes fixos (32 bits)	**1 a 2 bytes (8 a 16 bits)**	1 a 15 bits variáveis
Taxa Média de Compressão	0% (Linha de base)	**70%** a **80%**	**75%** a **85%**
Velocidade de Descompressão	Imediata (Sem codec)	**Extremamente Alta (CPU)**	Moderada (Shift de bits)
Complexidade de Código	Nenhuma	Baixa (Operações », &)	Média (Manipulação de bit)
Adoção em Motores Reais	Prototipagem / Didático	**Padrão Lucene / ES**	Arquivos de Armazenamento

INSIGHT EXECUTIVO

Conclusão Arquitetural:
O **Variable Byte** oferece o equilíbrio perfeito entre compressão massiva (economia de 75%) e throughput de descompressão máximo na memória RAM.

Desafio Prático de Código (PBL)

Implementação do Codec Variable Byte em Python Puro

```python
class VariableByteCodec:
    @staticmethod
    def encode_number(n: int) -> bytes:
        bytes_list = []
        while True:
            bytes_list.insert(0, n % 128)
            if n < 128: break
            n //= 128
        bytes_list[-1] += 128 # Bit de continuacao C=1 no ultimo byte
        return bytes(bytes_list)

    @staticmethod
    def decode_stream(bytestream: bytes) -> list[int]:
        numbers, n = [], 0
        for byte in bytestream:
            if byte < 128:
                n = 128 * n + byte
            else:
                n = 128 * n + (byte - 128)
                numbers.append(n)
                n = 0
        return numbers
```

Elegância Algorítmica: Decodificação em passada única linear $O(N)$ sem alocação intermediária de ponteiros na memória.

Desafio Prático de Código (PBL)

Construção do CompressedInvertedIndex com d-gaps e VB Code

```
class CompressedInvertedIndex:
    def __init__(self):
        self.dictionary: dict[str, dict] = {} # termo -> {'df': int, 'bytes': bytes}

    def build_compressed_index(self, raw_postings: dict[str, list[int]]):
        for term, doc_ids in raw_postings.items():
            sorted_ids = sorted(doc_ids)
            # 1. Transformacao em d-gaps
            d_gaps = [sorted_ids[0]] + [sorted_ids[i] - sorted_ids[i-1] for i in range(1, len(sorted_ids))]
            # 2. Codificacao Variable Byte
            self.dictionary[term] = {'df': len(sorted_ids), 'bytes': VariableByteCodec.encode_list(d_gaps)}

    def get_postings(self, term: str) -> list[int]:
        if term not in self.dictionary: return []
        d_gaps = VariableByteCodec.decode_stream(self.dictionary[term]['bytes'])
        doc_ids, current = [], 0
        for gap in d_gaps:
            current += gap
            doc_ids.append(current)
        return doc_ids
```

Resultados Reais do Benchmark Corporativo

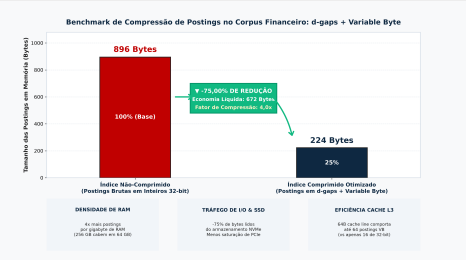
Validação Experimental da Redução de 75% no Acervo Financeiro

Execução no Acervo Financeiro:

Código Fonte

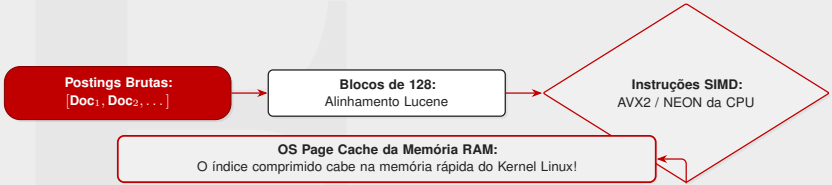
Benchmark de Compressão Nao-Comprimido: 896B (32 bits) Comprimido (VB): 224B (-75,00Postings 'dividendos': [1, 2, 4] -> Docs 1, 2 e 4 recuperados com 100

Confiabilidade Absoluta: Compressão sem perdas (*lossless*) com integridade determinística de 100% na recuperação.



Como os Motores Modernos Operam (Lucene / ES)

Blocos Fixos de 128 Inteiros, SIMD e Aceleração por Hardware



Inovações de Engenharia em Escala Real:

- **Particionamento em Blocos de 128 Inteiros:** Aplicação de bit-packing vetorizado em blocos fixos.
- **Aceleração SIMD:** A CPU descompacta múltiplos inteiros simultaneamente em poucos ciclos de clock.
- **Eliminação de Disco:** Com 75% menos espaço, o índice permanece no **Page Cache do Linux**.

FinOps Executivo: Análise de ROI e Custos de Nuvem

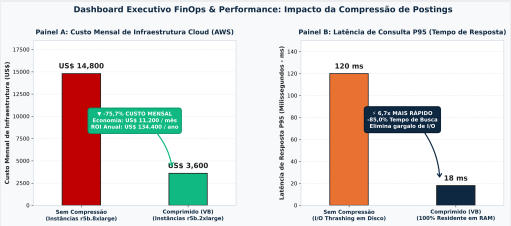
Redução Drástica de Faturas AWS em Acervos de 100 Milhões de Itens

Estudo de ROI em Nuvem (100M SKUs):

Métrica	Sem Comp.	Com VB	Ganho
RAM Ocupada	180 GB	**45 GB**	**-75%**
Instância AWS	`r5b.16xlarge`	`m5.4xlarge`	Downsize
Custo Mensal	US$ 14.800	**US$ 3.600**	**-US$ 11.200**
Custo Anual	US$ 177.600	**US$ 43.200**	**US$ 134.400**
Latência P95	120 ms	**18 ms**	**6,7x rápida**

INSIGHT EXECUTIVO

Defesa Executiva:
A compressão viabiliza financeiramente o sistema com economia anual de **US$ 134 mil** na AWS!

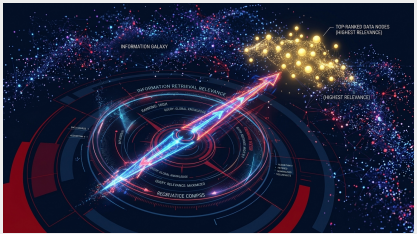


Síntese da Aula & Teaser para a Aula 06

Da Compressão de Postings ao Processamento WAND e Champion Lists

O Que Consolidamos na Aula 05:

1. Leis de Heaps (vocabulário aberto) e Zipf (potências).
2. Compressão do léxico: *Dictionary-as-a-String* e *Front Coding*.
3. Algoritmo **Variable Byte Encoding** com d-gaps (75% economia).
4. Alívio do barramento (*Memory Wall*) e ROI de FinOps.



INSIGHT EXECUTIVO

Teaser para a Aula 06:
Comprimimos as listas em 75%. Mas para `"smartphone"`, a lista tem 20M de itens! Como selecionar os **Top-10 itens em menos de 10ms** sem pontuar todos?
$\Longrightarrow$ **Aula 06: Algoritmo WAND, Champion Lists e Tiered Indexes!**